Jev market decisions: a paper simulation, latency checks and risk gates

Researched 2026-10-12.

Separate the API from the trading claim

The current TypeSafe quickstart (https://docs.typesafe.ai/introduction/quickstart) documents a state plus Choice, Score and Noul questions. It links to the console (https://console.typesafe.ai) for access. Python currently requires 3.10+ and the SDK is named `typesafe-sdk`; this is more reliable than guessing imports from the reel. Prices and latency figures in Muse's dossier were not reproduced here.

```
python -m pip install typesafe-sdk
```

Read the current quickstart's `TypeSafeClient`, `system_one`, `Choice`, `Score` and `Noul` example before adapting the interface. Store the key in `TYPESAFE_API_KEY`, never in a notebook, commit or public guide. Installation is documented; it has not been executed for this research.

A bounded research architecture

1. Record timestamped market snapshots with an explicit schema and stale-data threshold.
2. Implement a deterministic mock decision adapter first, so replay works without credentials.
3. Add Jev as an interchangeable decision adapter. Log wall-clock latency, timeouts, state digest and returned values.
4. Keep inventory limits, message-rate limits, stale-feed checks and cancel behavior deterministic outside the model.
5. Simulate fills with conservative assumptions, fees, spread and slippage. A favorable midpoint mark is not a realized fill.
6. Compare with a no-model baseline on held-out periods. Check whether probabilities are calibrated on that task; a confidence output is not a verified market probability.

What remains unproved

This check establishes the typed API interface, not a working exchange integration, a guaranteed sub-100ms loop, the supplied strategy's profitability or any live-order result. The source's "HFT" label and model-calibration claims need measurements in the intended environment. A networked API decision is only one component of total feed-to-order latency.

Proposed project

Build a replay-only simulator with no live order adapter or exchange trading keys. Require tests for delayed events, missing quotes, adapter failures, duplicate snapshots and a halt condition. Produce a latency histogram, calibration plot and net-of-cost comparison. These are research outputs, not a recommendation to deploy capital.

Reusable project prompt

```
Build Typed-decision replay simulator. Historical/replayed snapshots, mock and Jev adapters, deterministic risk gates; no live orders. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Define replay schema and baseline; Implement mock adapter and risk tests; Add documented Jev adapter; Measure latency and calibration.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://www.instagram.com/reel/DeAr43JPbqv/